Day 3

MCP · RAG 구현 & 외부 시스템 연동

AI Agent 전문 개발 과정

2026 · AI 개발자 · 데이터 엔지니어 · 기술 리더

오늘의 여정

오전

- Session 1: MCP 고급 설계 (2h)
- Session 2: 외부 API 최적화 (2h)

오후

- Session 3: RAG 4가지 요소 (2h)
- Session 4: Hybrid 아키텍처 (2h)

강의 30% · 실습 70% · 총 8시간

Session 1

MCP(Function Calling) 고급 설계

Tool 설계 품질 = Agent 신뢰도

❌ **모호한 Tool**
잘못된 호출

→

⚠️ **예측 불가**
결과

→

💥 **장애**
프로덕션

Anthropic 내부 데이터 (2026)

Agent 장애 원인 1위 = 잘못 설계된 Tool 스키마

Tool 정의 3요소

Tool 정의 3요소

name

Tool 식별자

```
get_current_weather
```

동사_명사 패턴

Tool 정의 3요소

name

Tool 식별자

```
get_current_weather
```

동사_명사 패턴

description

언제 쓸지 설명

```
"현재 날씨 요청 시. 예보는 forecast 사용."
```

조건 + 구분 명시

Tool 정의 3요소

name

Tool 식별자

```
get_current_weather
```

동사_명사 패턴

description

언제 쓸지 설명

```
"현재 날씨 요청 시. 예보는 forecast 사용."
```

조건 + 구분 명시

parameters

입력 스키마

```
type + description + examples + enum
```

필수/선택 구분

description: 나쁜 예 vs 좋은 예

나쁜 예

```
"Gets weather data"
```

- → 언제 쓰는지 불명확
- → 유사 Tool과 구분 불가
- → 모델이 잘못 선택

좋은 예

```
"사용자가 특정 도시의
현재 날씨를 물어볼 때 호출.
예보는 get_weather_forecast 사용."
```

- → 호출 조건 명시
- → 경쟁 Tool 구분
- → 정확한 선택

완전한 parameters 정의

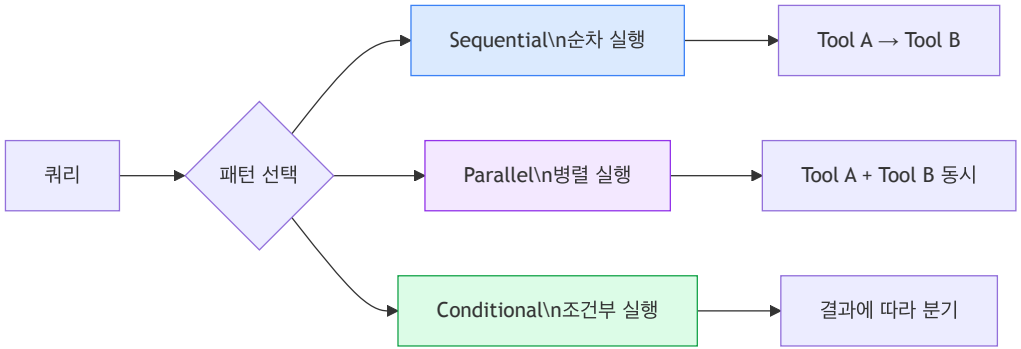
```
{
  "type": "object",
  "properties": {
    "city": {
      "type": "string",
      "description": "도시명 (영문 또는 한글)",
      "examples": ["Seoul", "서울", "Tokyo"]
    },
    "unit": {
      "type": "string",
      "enum": ["celsius", "fahrenheit"],
      "default": "celsius",
      "description": "온도 단위. 기본값: celsius"
    }
  },
  "required": ["city"]
}
```

완전한 parameters 정의

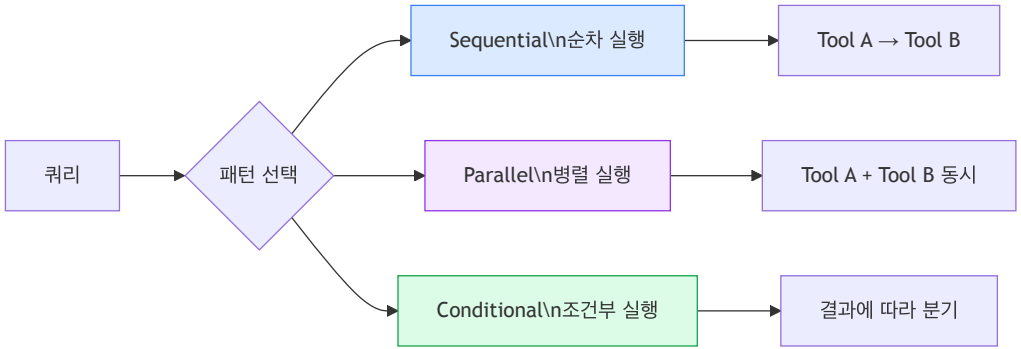
```
{
  "type": "object",
  "properties": {
    "city": {
      "type": "string",
      "description": "도시명 (영문 또는 한글)",
      "examples": ["Seoul", "서울", "Tokyo"]
    },
    "unit": {
      "type": "string",
      "enum": ["celsius", "fahrenheit"],
      "default": "celsius",
      "description": "온도 단위. 기본값: celsius"
    }
  },
  "required": ["city"]
}
```

enum	examples	required
허용값 제한	포맷 가이드	필수값 명시

Multi-tool Routing 3가지 패턴

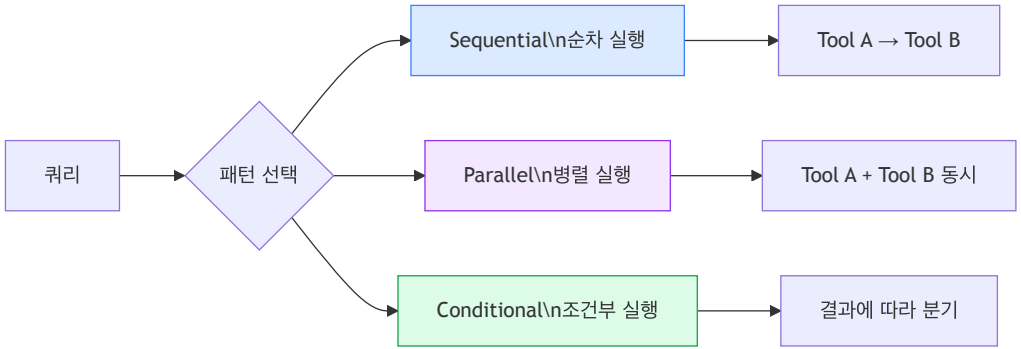


Multi-tool Routing 3가지 패턴



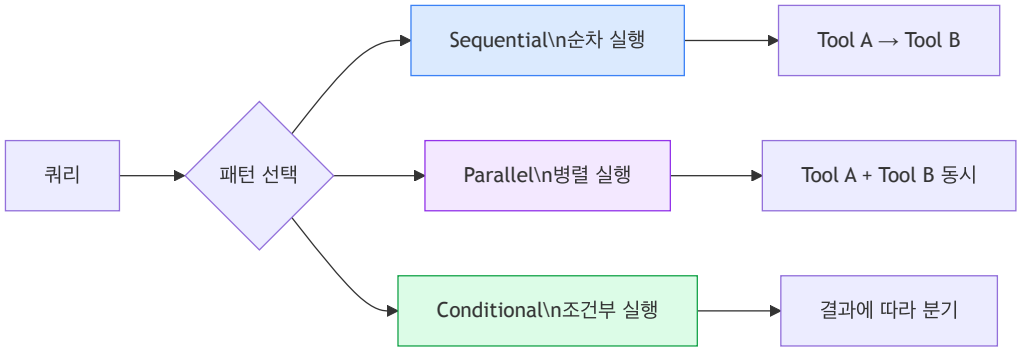
- **Sequential** → Tool A 결과를 Tool B 입력으로 사용

Multi-tool Routing 3가지 패턴



- **Sequential** → Tool A 결과를 Tool B 입력으로 사용
- **Parallel** → 독립 Tool을 동시 호출, 지연 단축

Multi-tool Routing 3가지 패턴



- **Sequential** → Tool A 결과를 Tool B 입력으로 사용
- **Parallel** → 독립 Tool을 동시 호출, 지연 단축
- **Conditional** → 첫 결과에 따라 다음 Tool 선택

Tool 실패 처리 전략

실패 유형별 처리

유형	처리
Timeout	지수 백오프 재시도
Invalid Input	오류 → 모델 재생성
Auth Error	즉시 중단
Rate Limit	대기 후 재시도
Not Found	Fallback Tool

오류 메시지 설계

```
{
  "type": "tool_result",
  "tool_use_id": "tool_123",
  "is_error": True,
  "content": "날씨 API 응답 시간 초과."
             " 잠시 후 재시도하거나"
             " 다른 방법으로 확인."
}
```

Tool 실패 처리 전략

실패 유형별 처리

유형	처리
Timeout	지수 백오프 재시도
Invalid Input	오류 → 모델 재생성
Auth Error	즉시 중단
Rate Limit	대기 후 재시도
Not Found	Fallback Tool

오류 메시지 설계

```
{
  "type": "tool_result",
  "tool_use_id": "tool_123",
  "is_error": True,
  "content": "날씨 API 응답 시간 초과."
             " 잠시 후 재시도하거나"
             " 다른 방법으로 확인."
}
```

핵심: 오류 메시지는 모델이 읽는다. 원인 + 대안을 구체적으로 작성해야 모델이 적절히 복구한다.

실습 1: Multi-tool Routing 정확도 비교

I DO (15분)

→ 모호한 Tool 정의 시연
→ 개선된 정의 비교
→ Parallel 호출 시연
→ Fallback 동작 확인

WE DO (30분)

→ 3개 Tool 정의
→ Agent 루프 구현
→ 병렬 호출 구현
→ Fallback 테스트

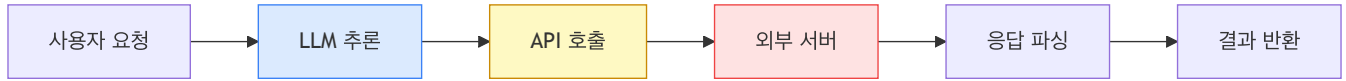
YOU DO (30분)

→ Tool 2개 추가
→ 정확도 측정 10케이스
→ Fallback 3 시나리오
→ solution/ 제출

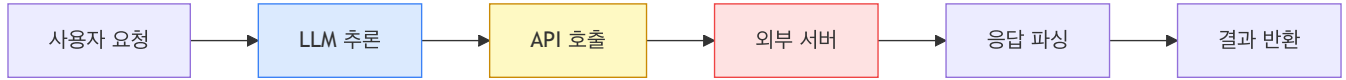
Session 2

외부 API · 데이터 연동 최적화

API 지연의 숨겨진 비용

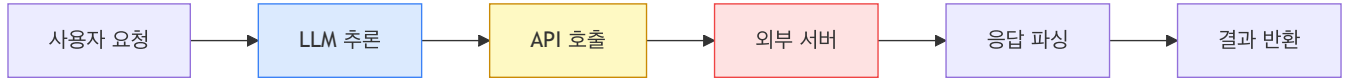


API 지연의 숨겨진 비용



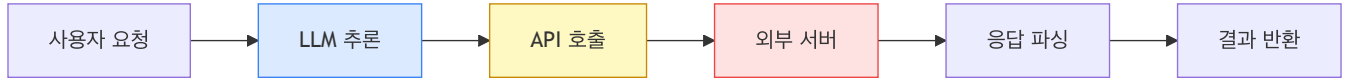
- **LLM 추론** → 줄이기 어려움. Tool 호출 횟수 최소화

API 지연의 숨겨진 비용



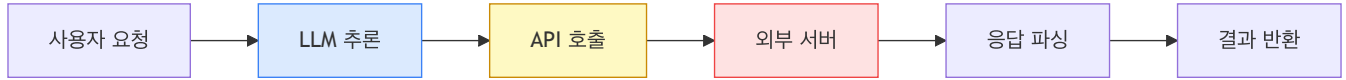
- **LLM 추론** → 줄이기 어려움. Tool 호출 횟수 최소화
- **API 호출** → HTTP/2, Connection Pool로 50~200ms 절감

API 지연의 숨겨진 비용



- **LLM 추론** → 줄이기 어려움. Tool 호출 횟수 최소화
- **API 호출** → HTTP/2, Connection Pool로 50~200ms 절감
- **외부 서버** → CDN, 엣지 서버로 100~500ms 절감

API 지연의 숨겨진 비용



- **LLM 추론** → 줄이기 어려움. Tool 호출 횟수 최소화
- **API 호출** → HTTP/2, Connection Pool로 50~200ms 절감
- **외부 서버** → CDN, 엣지 서버로 100~500ms 절감
- **캐싱** → 전체 파이프라인 생략으로 최대 절감

Connection Pool: 연결을 재사용하라

매번 새 연결 생성

```
# 나쁜 패턴
for query in queries:
    resp = requests.get(url)  # 매번 TCP 연결
    # 3-way handshake: +100~200ms
```

→ 요청당 100~200ms 낭비

Connection Pool 재사용

```
# 좋은 패턴
client = httpx.AsyncClient(
    limits=httpx.Limits(
        max_keepalive_connections=20
    )
)
# 연결 재사용 → 0ms
```

→ 연결 재사용으로 낭비 제거

동기 vs 비동기: 3배 차이

동기 순차 실행 — 3초

```
weather = requests.get(weather_url)  # 1초
news = requests.get(news_url)        # 1초
stocks = requests.get(stocks_url)    # 1초
# 합계: 3초
```

비동기 병렬 실행 — ~1초

```
weather, news, stocks = await asyncio.gather(
    client.get(weather_url),
    client.get(news_url),
    client.get(stocks_url),
    return_exceptions=True
)
# 합계: ~1초
```

동기 vs 비동기: 3배 차이

동기 순차 실행 — 3초

```
weather = requests.get(weather_url)  # 1초
news = requests.get(news_url)        # 1초
stocks = requests.get(stocks_url)    # 1초
# 합계: 3초
```

비동기 병렬 실행 — ~1초

```
weather, news, stocks = await asyncio.gather(
    client.get(weather_url),
    client.get(news_url),
    client.get(stocks_url),
    return_exceptions=True
)
# 합계: ~1초
```

return_exceptions=True: 하나 실패해도 나머지 결과 활용 가능

Schema Validation: Pydantic으로 검증

```
from pydantic import BaseModel, Field, validator

class WeatherRequest(BaseModel):
    city: str = Field(min_length=1, max_length=100)
    unit: str = Field(default="celsius",
                      pattern="^(celsius|fahrenheit)$")
    days: int = Field(default=1, ge=1, le=14)

    @validator("city")
    def sanitize_city(cls, v):
        forbidden = ["'", '"', ";", "DROP", "SELECT"]
        for char in forbidden:
            if char.upper() in v.upper():
                raise ValueError(f"허용되지 않는 문자: {char}")
        return v.strip()
```

Schema Validation: Pydantic으로 검증

```
from pydantic import BaseModel, Field, validator

class WeatherRequest(BaseModel):
    city: str = Field(min_length=1, max_length=100)
    unit: str = Field(default="celsius",
                      pattern="^(celsius|fahrenheit)$")
    days: int = Field(default=1, ge=1, le=14)

    @validator("city")
    def sanitize_city(cls, v):
        forbidden = ["'", '"', ";", "DROP", "SELECT"]
        for char in forbidden:
            if char.upper() in v.upper():
                raise ValueError(f"허용되지 않는 문자: {char}")
        return v.strip()
```

- 입력 검증: SQL Injection, XSS 방어

Schema Validation: Pydantic으로 검증

```
from pydantic import BaseModel, Field, validator

class WeatherRequest(BaseModel):
    city: str = Field(min_length=1, max_length=100)
    unit: str = Field(default="celsius",
                      pattern="^(celsius|fahrenheit)$")
    days: int = Field(default=1, ge=1, le=14)

    @validator("city")
    def sanitize_city(cls, v):
        forbidden = ["'", '"', ";", "DROP", "SELECT"]
        for char in forbidden:
            if char.upper() in v.upper():
                raise ValueError(f"허용되지 않는 문자: {char}")
        return v.strip()
```

- 입력 검증: SQL Injection, XSS 방어
- 출력 검증: 응답 형식 및 값 범위 확인

Schema Validation: Pydantic으로 검증

```
from pydantic import BaseModel, Field, validator

class WeatherRequest(BaseModel):
    city: str = Field(min_length=1, max_length=100)
    unit: str = Field(default="celsius",
                      pattern="^(celsius|fahrenheit)$")
    days: int = Field(default=1, ge=1, le=14)

    @validator("city")
    def sanitize_city(cls, v):
        forbidden = ["'", '"', ";", "DROP", "SELECT"]
        for char in forbidden:
            if char.upper() in v.upper():
                raise ValueError(f"허용되지 않는 문자: {char}")
        return v.strip()
```

- 입력 검증: SQL Injection, XSS 방어
- 출력 검증: 응답 형식 및 값 범위 확인
- 실패 시: 구조화된 오류 메시지 반환

인증 · 보안 · Rate Limit

인증 · 보안 · Rate Limit

API 키 관리

- ❌ 코드에 하드코딩
- ❌ Git에 config.json
- ✅ 환경변수
- ✅ Secret Manager

인증 · 보안 · Rate Limit

API 키 관리

- ❌ 코드에 하드코딩
- ❌ Git에 config.json
- ✅ 환경변수
- ✅ Secret Manager

Timeout 설정

```
connect: 2.0s
read: 10.0s
write: 5.0s
```

각각 설정 필수

인증 · 보안 · Rate Limit

API 키 관리

- ❌ 코드에 하드코딩
- ❌ Git에 config.json
- ✅ 환경변수
- ✅ Secret Manager

Timeout 설정

```
connect: 2.0s
read: 10.0s
write: 5.0s
```

각각 설정 필수

429 처리

Retry-After 헤더 확인

해당 시간 대기

재시도 실행

실습 2: API 연동 검증 로직 비교

I DO (15분)

→ 동기 vs 비동기 시간 비교
→ 검증 없을 때 오류
→ Pydantic 검증 추가

WE DO (30분)

→ AsyncClient 구현
→ TTL 캐시 추가
→ Pydantic 모델 작성
→ Rate Limiter 구현

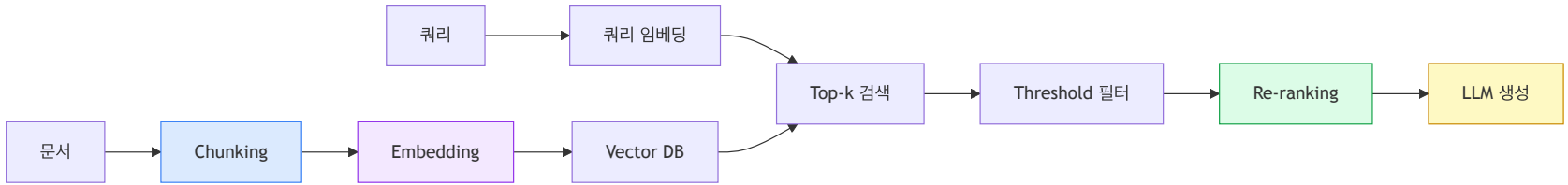
YOU DO (30분)

→ 3개 API 병렬 클라이언트
→ 캐시 + Rate Limiter 통합
→ 오류 유형별 처리

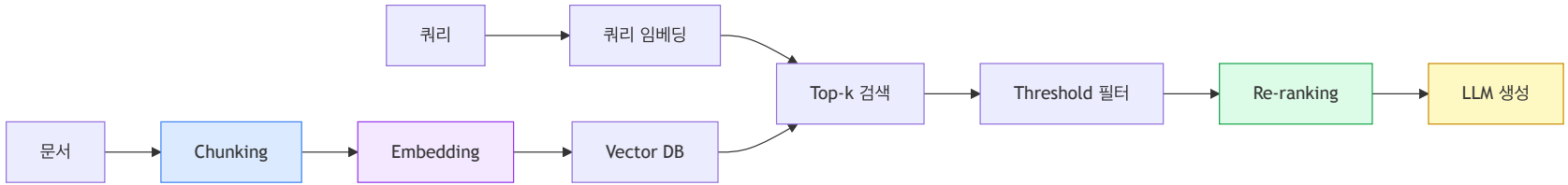
Session 3

RAG 성능을 결정하는 4가지 요소

RAG 파이프라인 전체 구조

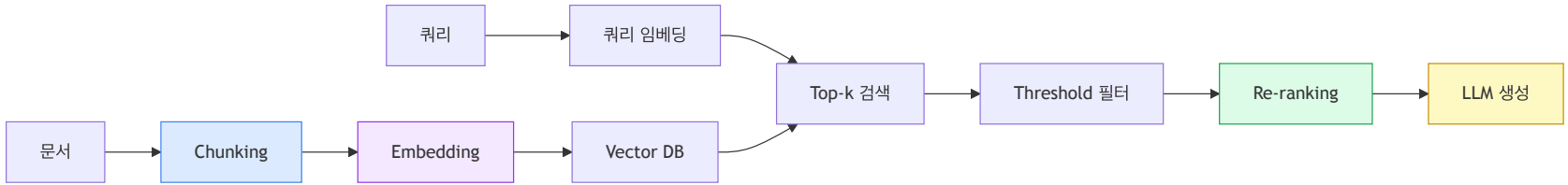


RAG 파이프라인 전체 구조



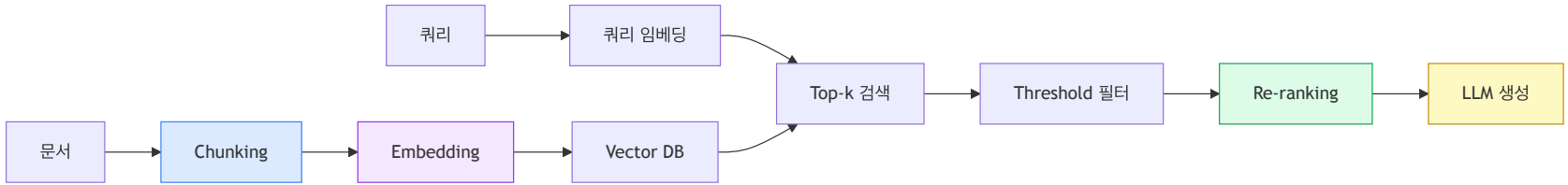
- **1단계** Chunking → 검색 단위 분할

RAG 파이프라인 전체 구조



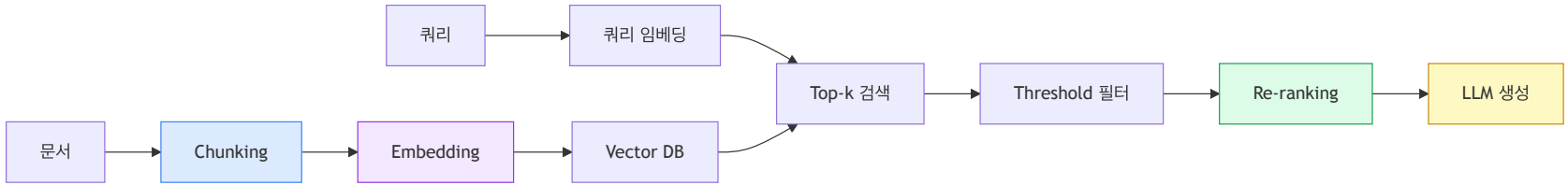
- **1단계** Chunking → 검색 단위 분할
- **2단계** Embedding → 의미 벡터 변환

RAG 파이프라인 전체 구조



- **1단계** Chunking → 검색 단위 분할
- **2단계** Embedding → 의미 벡터 변환
- **3단계** Retrieval → Top-k + Threshold + Re-ranking

RAG 파이프라인 전체 구조



- **1단계** Chunking → 검색 단위 분할
- **2단계** Embedding → 의미 벡터 변환
- **3단계** Retrieval → Top-k + Threshold + Re-ranking
- **4단계** Guardrail → Hallucination 방지

Chunking: 4가지 전략

Chunking: 4가지 전략

Fixed-size

512 토큰 고정 분할

비구조화 텍스트

Chunking: 4가지 전략

Fixed-size

512 토큰 고정 분할

비구조화 텍스트

Semantic

문장·단락 경계 분할

일반 문서, 기사

Chunking: 4가지 전략

Fixed-size

512 토큰 고정 분할

비구조화 텍스트

Semantic

문장·단락 경계 분할

일반 문서, 기사

Document-aware

헤더·구조 단위 분할

Markdown, HTML

Chunking: 4가지 전략

Fixed-size
512 토큰 고정 분할
비구조화 텍스트

Semantic
문장·단락 경계 분할
일반 문서, 기사

Document-aware
헤더·구조 단위 분할
Markdown, HTML

Hierarchical
대단원 → 소단원 → 문장
복잡한 기술 문서

Chunking: 4가지 전략

Fixed-size
512 토큰 고정 분할
비구조화 텍스트

Semantic
문장·단락 경계 분할
일반 문서, 기사

Document-aware
헤더·구조 단위 분할
Markdown, HTML

Hierarchical
대단원 → 소단원 → 문장
복잡한 기술 문서

Chunk 크기 기준: 너무 크면 노이즈, 너무 작으면 맥락 손실 → **512±100 토큰**이 일반 출발점

Embedding 모델 선택 기준 (2026)

모델	차원	한국어	비용	추천 상황
text-embedding-3-large	3072	우수	유료	프로덕션 (영어 중심)
text-embedding-3-small	1536	양호	저렴	비용 민감
multilingual-e5-large	1024	**최우수**	무료	한국어 전용
bge-m3	1024	우수	무료	온프레미스
voyage-3	1024	양호	유료	Claude 최적화

Embedding 모델 선택 기준 (2026)

모델	차원	한국어	비용	추천 상황
text-embedding-3-large	3072	우수	유료	프로덕션 (영어 중심)
text-embedding-3-small	1536	양호	저렴	비용 민감
multilingual-e5-large	1024	**최우수**	무료	한국어 전용
bge-m3	1024	우수	무료	온프레미스
voyage-3	1024	양호	유료	Claude 최적화

한국어 서비스: `multilingual-e5-large` 또는 `bge-m3` 우선 고려
Claude 연동: Anthropic 권장 `voyage-3` 시리즈

Retrieval 튜닝: Top-k · Threshold · Re-ranking

Top-k

k=3 → 정확하지만 누락 위험

k=5~10 → 일반 QA 균형

k=20+ → 포괄 요약용

Threshold

0.65~0.75 권장 범위

높으면 → 누락 증가

낮으면 → 노이즈 증가

Re-ranking

Cross-encoder 사용

정확도 20~40% 향상

속도 5~10배 느림

Retrieval 튜닝: Top-k · Threshold · Re-ranking

Top-k

k=3 → 정확하지만 누락 위험

k=5~10 → 일반 QA 균형

k=20+ → 포괄 요약용

Threshold

0.65~0.75 권장 범위

높으면 → 누락 증가

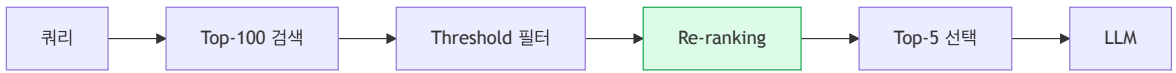
낮으면 → 노이즈 증가

Re-ranking

Cross-encoder 사용

정확도 20~40% 향상

속도 5~10배 느림



Hallucination 최소화 전략

원인과 해결책

원인	해결
관련 문서 없음	Threshold 강화
노이즈 문서	Re-ranking
프롬프트 부재	명시적 지침
컨텍스트 과다	Top-k 줄이기

핵심 프롬프트 패턴

```
SYSTEM = """
[참조 문서]에 있는 내용만 사용.
문서에 없으면 '확인 불가'라고 명시.
추측하지 말 것.
출처는 [출처: n]으로 표시.
"""
```

Hallucination 최소화 전략

원인과 해결책

원인	해결
관련 문서 없음	Threshold 강화
노이즈 문서	Re-ranking
프롬프트 부재	명시적 지침
컨텍스트 과다	Top-k 줄이기

핵심 프롬프트 패턴

```
SYSTEM = """
[참조 문서]에 있는 내용만 사용.
문서에 없으면 '확인 불가'라고 명시.
추측하지 말 것.
출처는 [출처: n]으로 표시.
"""
```

신뢰도 점수 < 0.6 → 답변 거부: "신뢰할 수 있는 정보를 찾을 수 없습니다"

실습 3: Chunk 전략별 Retrieval 비교

I DO (15분)

→ Fixed vs Semantic 비교
→ Chunk 크기 분포 확인
→ 동일 쿼리 결과 비교
→ Threshold 차이 시연

WE DO (30분)

→ 3가지 Chunking 구현
→ 벡터 스토어 인덱싱
→ 10개 쿼리 정확도 측정
→ Re-ranking 적용

YOU DO (30분)

→ 자체 문서로 RAG 구축
→ 전략 2가지 비교 실험
→ Threshold 3가지 비교
→ Faithfulness 측정

Session 4

Hybrid 아키텍처 설계

MCP vs RAG: 무엇이 다른가

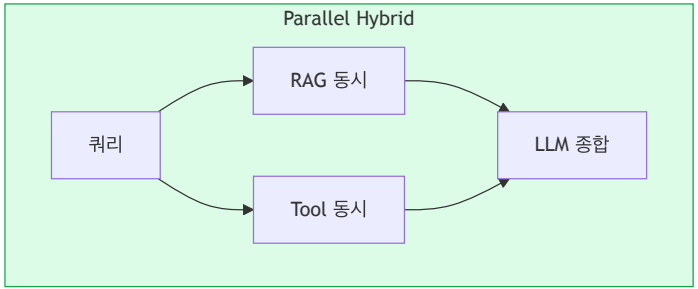
MCP (Tool Calling)

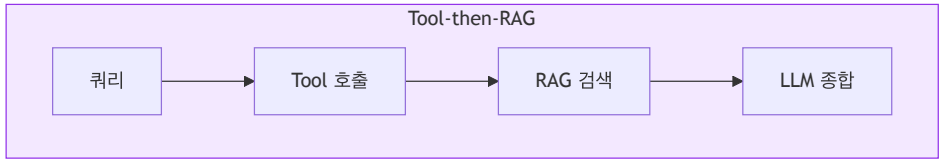
- ✅ 실시간 데이터
- ✅ 부작용 있는 행동
- ✅ 계산·변환
- ❌ 대용량 지식
- ❌ 비용 효율

RAG

- ✅ 대용량 지식 검색
- ✅ 저비용
- ✅ 빠른 검색
- ❌ 실시간 데이터
- ❌ 행동 수행

Hybrid 아키텍처 패턴 3가지





아키텍처 선택 기준

요구사항별 선택

요구사항	선택
실시간 + 대용량 지식	**Hybrid**
실시간만	MCP 중심
지식만, 저비용	RAG 중심
빠른 응답	RAG 중심
복잡한 추론	Hybrid

의사결정 질문

```
1. 실시간 데이터 필요?
   → 예: MCP 또는 Hybrid

2. 10,000개 이상 문서?
   → 예: RAG 또는 Hybrid

3. 부작용 있는 행동?
   → 예: MCP 또는 Hybrid

4. API 비용 최소화?
   → 예: RAG 중심
```

정확도 · 비용 · 확장성 Trade-off

정확도

LLM Only → 낮음

RAG → 중간

MCP → 중간

Hybrid → 최고

비용

RAG → 최저

LLM Only → 중간

MCP → 높음

Hybrid → 최고

확장성

RAG → 문서 무제한

MCP → API 한도 의존

Hybrid → 높음

LLM Only → 낮음

정확도 · 비용 · 확장성 Trade-off

정확도

LLM Only → 낮음

RAG → 중간

MCP → 중간

Hybrid → 최고

비용

RAG → 최저

LLM Only → 중간

MCP → 높음

Hybrid → 최고

확장성

RAG → 문서 무제한

MCP → API 한도 의존

Hybrid → 높음

LLM Only → 낮음

캐싱 적용 시 Hybrid 비용을 RAG 수준으로 낮출 수 있다

Hybrid Agent 핵심 로직

```python
async def run(self, query: str) → dict:
    # 1. RAG 먼저 (빠르고 저렴)
    rag_docs, rag_score = await self.rag.retrieve(query)

    # 2. 신뢰도에 따라 분기
    if rag_score ≥ 0.70:
        system = f"지식 기반 답변:\n{format_docs(rag_docs)}"
        # Tool 필요 시 추가 호출
    else:
        system = "실시간 데이터로 답변하세요."
        # Tool로 데이터 수집

    # 3. LLM + Tool 루프 실행
    response = await llm_with_tools(system, query)

    return {
        "answer": response,
        "architecture": classify(rag_score, tools_used)
    }
```

Hybrid Agent 핵심 로직

```python
async def run(self, query: str) → dict:
    # 1. RAG 먼저 (빠르고 저렴)
    rag_docs, rag_score = await self.rag.retrieve(query)

    # 2. 신뢰도에 따라 분기
    if rag_score ≥ 0.70:
        system = f"지식 기반 답변:\n{format_docs(rag_docs)}"
        # Tool 필요 시 추가 호출
    else:
        system = "실시간 데이터로 답변하세요."
        # Tool로 데이터 수집

    # 3. LLM + Tool 루프 실행
    response = await llm_with_tools(system, query)

    return {
        "answer": response,
        "architecture": classify(rag_score, tools_used)
    }
```

RAG 신뢰도 높음	RAG + Tool 모두	RAG 신뢰도 낮음
RAG-only	**Hybrid**	**Tool-only**

프로덕션 Hybrid 설계 원칙

프로덕션 Hybrid 설계 원칙

- **원칙 1** → RAG를 먼저 시도한다 (저비용 경로 우선)

프로덕션 Hybrid 설계 원칙

- **원칙 1** → RAG를 먼저 시도한다 (저비용 경로 우선)
- **원칙 2** → RAG 신뢰도 낮을 때만 Tool 호출

프로덕션 Hybrid 설계 원칙

- **원칙 1** → RAG를 먼저 시도한다 (저비용 경로 우선)
- **원칙 2** → RAG 신뢰도 낮을 때만 Tool 호출
- **원칙 3** → 실시간 데이터는 항상 Tool 사용

프로덕션 Hybrid 설계 원칙

- **원칙 1** → RAG를 먼저 시도한다 (저비용 경로 우선)
- **원칙 2** → RAG 신뢰도 낮을 때만 Tool 호출
- **원칙 3** → 실시간 데이터는 항상 Tool 사용
- **원칙 4** → 병렬 처리로 지연 최소화

프로덕션 Hybrid 설계 원칙

- **원칙 1** → RAG를 먼저 시도한다 (저비용 경로 우선)
- **원칙 2** → RAG 신뢰도 낮을 때만 Tool 호출
- **원칙 3** → 실시간 데이터는 항상 Tool 사용
- **원칙 4** → 병렬 처리로 지연 최소화
- **원칙 5** → 결과 캐싱으로 중복 호출 방지

프로덕션 Hybrid 설계 원칙

- **원칙 1** → RAG를 먼저 시도한다 (저비용 경로 우선)
- **원칙 2** → RAG 신뢰도 낮을 때만 Tool 호출
- **원칙 3** → 실시간 데이터는 항상 Tool 사용
- **원칙 4** → 병렬 처리로 지연 최소화
- **원칙 5** → 결과 캐싱으로 중복 호출 방지

모니터링 핵심 지표

아키텍처 비율	응답 지연	쿼리당 비용
RAG/Tool/Hybrid %	P50, P95, P99	RAG + Tool + LLM

실습 4: Hybrid 아키텍처 설계 및 구현

I DO (20분)

→ RAG-only vs MCP-only 비교
→ Hybrid 자동 전환 시연
→ Parallel Hybrid 지연 비교

WE DO (40분)

→ HybridAgent 클래스 구현
→ 신뢰도 기반 분기 로직
→ 병렬 실행 구현
→ 아키텍처 로깅 추가

YOU DO (45분)

→ 3유형 쿼리 세트 설계
→ 벤치마크 스크립트 작성
→ Trade-off 기준표 작성

Day 3 핵심 정리

Day 3 핵심 정리

Session 1: MCP 설계

description 조건 명시 → 선택 정확도 15~30% 향상

Day 3 핵심 정리

Session 1: MCP 설계

description 조건 명시 → 선택 정확도 15~30% 향상

Session 2: API 최적화

비동기 + 캐시 + Pydantic + Rate Limit = 프로덕션 기본

Day 3 핵심 정리

Session 1: MCP 설계

description 조건 명시 → 선택 정확도 15~30% 향상

Session 2: API 최적화

비동기 + 캐시 + Pydantic + Rate Limit = 프로덕션 기본

Session 3: RAG 4요소

Chunking → Embedding → Retrieval → Guardrail

Day 3 핵심 정리

Session 1: MCP 설계

description 조건 명시 → 선택 정확도 15~30% 향상

Session 2: API 최적화

비동기 + 캐시 + Pydantic + Rate Limit = 프로덕션 기본

Session 3: RAG 4요소

Chunking → Embedding → Retrieval → Guardrail

Session 4: Hybrid 설계

RAG 먼저 → 신뢰도 낮으면 Tool → 병렬 + 캐싱

내일(Day 4) 예고

오전

→ Agent 평가 프레임워크 설계

→ 프로덕션 모니터링 · 관찰 가능성

오후

→ Agent 성능 최적화 전략

→ 비용 최적화 실전 기법

오늘 실습 코드를 복습하면서 Hybrid Agent를 직접 확장해 보세요.

Q&A

Day 3 질문을 받겠습니다

MCP 설계 · API 연동 · RAG · Hybrid 아키텍처

수고하셨습니다

내일 Day 4에서 뵙겠습니다